Chapter 1

Finding Shortest Paths

1.1 Introduction — BFS revisited

We have seen BFS as a way to search graphs (or trees). It is a pretty simple process:

- select a start node
- move one distance from the root, and visit all nodes (v) on that level
- when visiting, queue all child nodes (v') of that node.
- keep going until either all nodes are marked *visited* or a provided *target node* is reached.

There is an issue here: what if the graph is weighted, and the paths are not equally weighted? For example, see this graph:

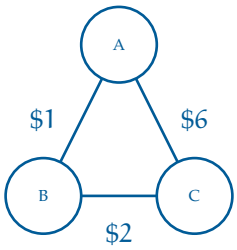


Figure 1.1: A graph of 3 nodes that would fail BFS.

BFS, here, would prefer $A \to C$, since it has an edge count of 1, whereas $A \to B \to C$ has an edge count of 2. However, the true shortest path, by weight, is $A \to B \to C$, with a weight of 3.

So BFS has a major limitation: it cannot handle weighted graphs. This is why we started it out on trees; typically trees do not have a set edge weight, so we were not limited by the order in which it expands.

To handle weighted graphs, we use a new algorithm that expands off of BFS, called *Dijkstra's Algorithm*. Dijkstra's Algorithm avoids the limitation of weights that BFS had by enqueueing the nodes in order of current shortest known distance.

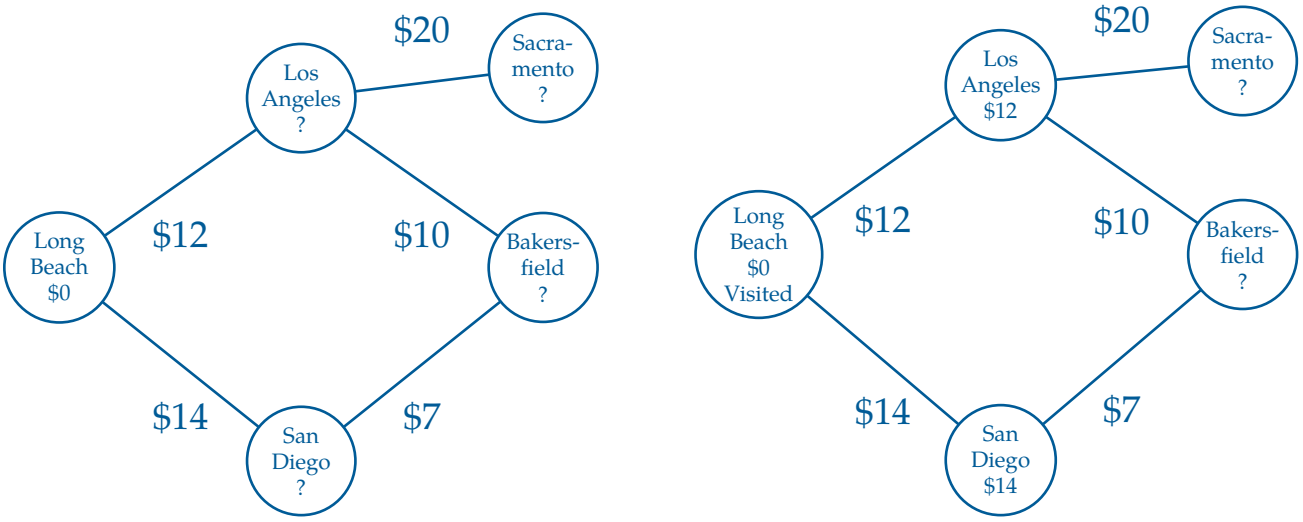
1.2 Dijkstra's Algorithm

Edsger W. Dijkstra (pronounced *DYKE-strah*) was a great Dutch computer scientist. He came up with an algorithm for finding the cheapest path through a graph with weighted edges. Today it is known as *Dijkstra's Algorithm*. It is used in a wide variety of common problems, and is also both simple and elegant.

1.2.1 Algorithm Description

You are going to mark each node with how much it would cost to get there from some chosen origin node. For example, if you are shipping a container from Long Beach, you will mark each city with the cost of getting the container to that city.

You start by marking the price for Long Beach to zero (the container is already there). You then mark each adjacent city with the cost on the edge (figure 1.2a). Next, you declare Long Beach to be "visited" (figure 1.2b).



(a) The initialized state of the graph, selecting Long Beach as our origin node.

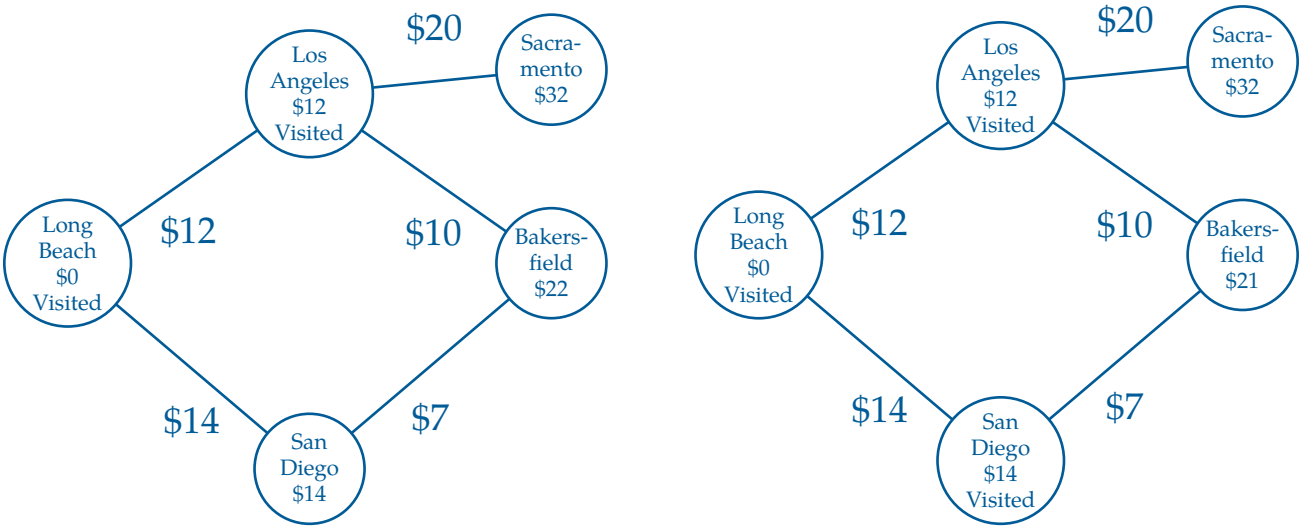
(b) Marking Long Beach as visited, we change the cost values of Los Angeles and San Diego to their respective edges.

Figure 1.2: A map of weighted cities.

After that, you find the cheapest of the unvisited nodes. In this case, Los Angeles is cheaper than San Diego, so that is the node you will visit next.

You mark all of the unvisited nodes adjacent to Los Angeles, with the price to ship it to Los Angeles plus the cost of shipping the container from Los Angeles to that city (figure

1.3a). Note that Bakersfield is marked with $22.



(a) After visiting Los Angeles.

(b) After visiting San Diego.

Figure 1.3: Visiting the next two minimum cost cities.

Now, the cheapest unvisited node is San Diego, so you mark its neighbors with the cost to ship to San Diego, plus the price to ship from San Diego to the neighbor (figure 1.3b).

Notice that Bakersfield is already labeled with $22 from a route through Los Angeles. But the price would be $21 if you shipped it to Bakersfield via San Diego. Because the new route is cheaper, you change the price to the lower value.

(What does it mean that a node is "visited"? If a node is marked visited, it is marked with a price that won't get any smaller.)

You continue visiting the cheapest unvisited node until all the nodes have been visited. Then you know every node has been marked with its lowest price.

In a big graph, each node may be marked several times in this process — each time with a lower price from a cheaper router.

Of course, once you have the price, you will ask, "What is the route that gets me that price?" So, we will also mark each node with the neighbor from which it would receive the shipment — the previous node. This is easy to do as we execute the algorithm.

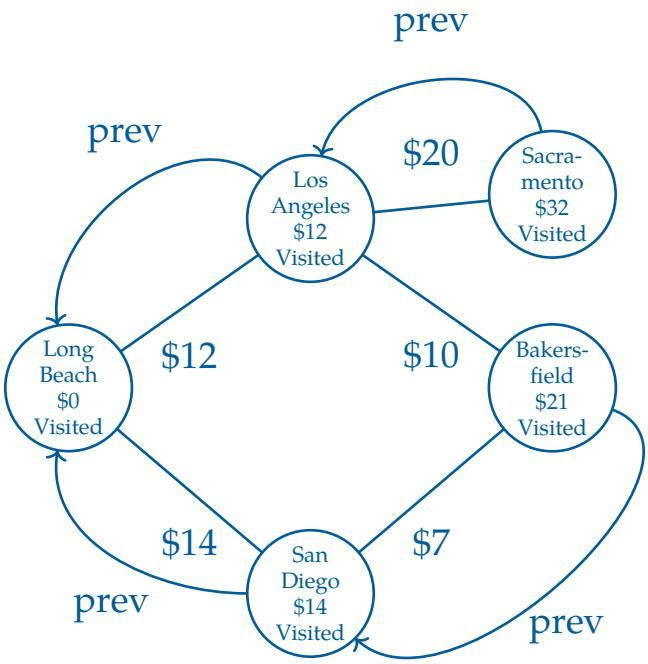


Figure 1.4: Marking each node with a previous pointer.

Now, to figure out the cheapest route from Long Beach to Bakersfield, we start at the destination and follow the `prev` pointer back through San Diego, then to Long Beach.

Exercise 1 Expanding on the Delivery Graph

Notice that the path between Los Angeles to Bakersfield is not used once we add in our previous labels. Why might this be?

Working Space

Answer on Page 11

1.2.2 Implementation

We do not actually want to sully our graph objects with the three additional pieces of information we need:

- The current minimal cost from the origin node. This is usually called the `dist`, from "distance".
- The neighbor who gives us the current minimal cost. This is usually called `prev`, from "previous".
- Whether the city is visited or now.

So, we will keep them in collections external to the graph.

For example, to keep track of the `dist`, we will have a `dist` dictionary. Each node will be a key, the current minimal cost will be the value. If the node hasn't received even a first cost, we will put in infinity as the cost.

(After the algorithm is run, if the cost of a node is still infinity, that means that it cannot be reached from the origin node.)

We will also have a `prev` dictionary. The final node will be the key, and its previous neighbor will be the value.

Finally, the graph has a list of all the nodes, so we can just keep a set of the unvisited nodes.

Add a method to the `Graph` class that implements Dijkstra's algorithm:

```
    def cost_from_node(self, origin_node):
        # Cost of cheapest path from origin node discovered so far
        # Initially the origin is zero and all the other are infinity
        dist = {k: math.inf for k in self.nodes}
        dist[origin_node] = 0.0

        # The previous city on that cheapest path
        prev = {}

        # All the nodes start as unvisited
        unvisited = set(self.nodes)

        # While there are still unvisited nodes
        while unvisited:

            # Find unvisited node with lowest cost
            min_cost = math.inf
```

```
18              for u in unvisited:
19                  if dist[u] < min_cost:
20                      current_node = u
21                      min_cost = dist[u]
22
23              # If none are less than inf, we are done
24              # This happens in graphs that are not connected
25              if min_cost == math.inf:
26                  return (dist, prev)
27
28              # Remove the lowest cost node from the unvisited list
29              unvisited.remove(current_node)
30
31              # Update all the unvisited neighbors
32              for edge in current_node.edges:
33
34                  # What node is at the other end of this edge?
35                  v = edge.other_end(current_node)
36
37                  # Visited nodes are already minimized, skip them
38                  if v not in unvisited:
39                      continue
40
41                  # Is this a shorter route?
42                  alt = dist[current_node] + edge.cost
43                  if alt < dist[v]:
44
45                      # Update the distance and prev dicts
46                      dist[v] = alt
47                      prev[v] = current_node
48
49          return (dist, prev)
```

Each time we visit a node, we have a decision to make:

$$\text{if } \text{dist}[u] + w(u, v) < \text{dist}[v] \text{ then update } \text{dist}[v]$$

This action, the core foundation of Dijkstra's, is done in lines 16-21. In Dijkstra's, the node with the minimum tentative distance can also be finalized, provided all edge weights are non-negative. Nodes left at infinity are not reachable from the chosen root node.

Append some code to your `cities.py` that tests this method:

```
1  (cost_from_long_beach, prev) = network.cost_from_node(long_beach)
2  print(f"\nMinimum costs from Long Beach = {cost_from_long_beach}")
3  print(f"\nLast city before = {prev}")
4
5  nyc_cost = cost_from_long_beach[nyc]
6
7  if nyc_cost < math.inf:
```

```
    print(f"\n*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")
else:
    print("You can't get to NYC from Long Beach")
```

When you run it, you should get a list of how much it costs to ship a container to each city from Long Beach:

```
Minimum costs from Long Beach = {(node:Long Beach, edges:1): 0.0,
(node:Los Angeles, edges:3): 12.0, (node:Denver, edges:3): 35.0,
(node:Phoenix, edges:3): 31.0, (node:Louisville, edges:3): 49.0,
(node:Cleveland, edges:4): 44.0, (node:Boston, edges:2): 57.0,
(node:New York City, edges:3): 52.0}
```

You will also get a collection of node pairs. What are these? For each node, you get the node that you would pass through on the cheapest route from Long Beach:

```
Last city before = {(node:Los Angeles, edges:3):(node:Long Beach, edges:1),
(node:Denver, edges:3):(node:Los Angeles, edges:3),
(node:Phoenix, edges:3):(node:Los Angeles, edges:3),
(node:Louisville, edges:3):(node:Denver, edges:3),
(node:Cleveland, edges:4):(node:Denver, edges:3),
(node:New York City, edges:3):(node:Cleveland, edges:4),
(node:Boston, edges:2): (node:Cleveland, edges:4)}
```

Your users will not want to read this; give them the shortest path as a list. Add a function to `graph.py` that turns the `prev` table into a path of nodes that lead from the origin to the destination:

```
def shortest_path(prev, destination):

    # Include the destination in the path
    path = [destination]
    current_node = destination

    # Keep stepping backward in the path
    while current_node in prev:

        # What node should come before the current node?
        previous_node = prev[current_node]

        # Insert it at the start of the list
        path.insert(0, previous_node)
        current_node = previous_node

    return path
```

Test that out:

```
if nyc_cost < math.inf:
    print(f"*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")

    path_to_nyc = graph.shortest_path(prev, nyc)
    print(f"*** Cheapest path from Long Beach to NYC: {path_to_nyc ***}")
else:
    print("You can't get to NYC from Long Beach")
```

This should look like this:

```
*** Cheapest path from Long Beach to NYC: [(node:Long Beach, edges:1),
(node:Los Angeles, edges:3), (node:Denver, edges:3), (node:Cleveland, edges:4),
(node:New York City, edges:3)] ***
\end{text}

% \subsection{Making it faster}

% On really big networks, doing a full Dijkstra's algorithm would take
% too long; luckily, there are many methods for getting similar results
% quickly.  When you ask for directions from Google Maps, it does not do
% a full Dijkstra's Algorithm for every possible route --- it would just
% take too long.

% However, there is a way to speed up this implementation. Look at this snippet:

% \begin{minted}{python}
%             # Find unvisited node with lowest cost
%             min_cost = math.inf
%             for u in unvisited:
%                 if dist[u] < min_cost:
%                     current_node = u
%                     min_cost = dist[u]
```

1.3 Bellman-Ford Algorithm

The Bellman-Ford Algorithm is another shortest path algorithm like Dijkstra's. However, unlike Dijkstra's Algorithm, Bellman-Ford can handle graphs with negative weight edges.

The algorithm works as follows:

1. Assign a tentative distance value for every node: set it to zero for our initial node and to infinity for all other nodes.
2. For each edge (u, v) with weight w, if the current distance to v is greater than the

distance to u plus w, update the distance to v to be the distance to u plus w.

3. Repeat the previous step $|V| - 1$ times, where $|V|$ is the number of vertices in the graph.
4. After the above steps, if you can still find a shorter path, there exists a negative cycle.

If the graph does not contain a negative cycle reachable from the source, the shortest paths are well-defined, and Bellman-Ford will correctly calculate them. If a negative cycle is reachable, no solution exists, but Bellman-Ford will detect it.

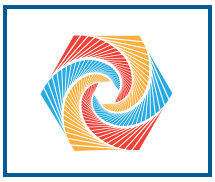
Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 4)

Los Angeles $\rightarrow$ Bakersfield is not chosen because it does not give the best known cost to Bakersfield. The `prev` pointer for Bakersfield goes to San Diego instead, since \$21 $<$ \$22.

The tree that is generated is called a *predecessor tree* or *explored tree* of the resulting edges formed by only the previous edges. We can use this to track shortest paths from a given origin node.



Index